Introduction to Software Engineering

Project Proposal & Plan

The student team is required to complete the **Project Proposal & Plan** documentation for the assigned course project, following the attached template.



Software Engineering Department
Faculty of Information and Technology
University of Science

Table of Contents

Objectives 1
1 Member Contribution Assessment 2
2 Vision 3
2.1 Preliminary Problem Statement 3
2.2 Product Position Statement 3
2.3 Stakeholders and Users 4
2.4 Scope and Key Features 4
3 Proposed Solution 6
3.1 Preliminary Architecture 6
3.2 Technology Stack 6
3.2.1 Core Technology Stack Summary 6
3.2.2 ORM Comparison Matrix 7
3.2.3 Hosting & Deployment Comparison Matrix 8
3.2.4 Testing Framework Comparison Matrix 9
3.3 Hardware and Operating Environment 10
4 Project Plan 12
4.1 Process and Phase Gates 12
4.2 Work Breakdown Structure (WBS) 14
4.3 Estimation 16
4.4 Schedule 22
4.5 Roles and Responsibilities 24
4.6 Human Resources and Cost Estimate 24
5 Risk Management 25
6 Tools & Collaboration Setup 27
7 AI Usage Log 29

Project Proposal & Plan

Objectives

This document focus on the following topics:

- ✔ Completing the Project Proposal & Plan document with the following sections:
  - Vision
  - Proposed Solution
  - Project Plan
  - Risk Management
  - Tools & Collaboration Setup
- ✔ Including an AI usage log for this phase
- ✔ Understanding the Project Proposal & Plan document.

1 Member Contribution Assessment

ID	Name	Role(s)	Contribution (%)	Evidence
24127462	Huỳnh Trung Ngạn	PM	25%	Build a work breakdown structure (WBS) Define project phases and exit criteria Set up communicationchannels Set up task board
24127175	Lê Đoàn Nhật Huy	BA	25%	Define the problem Identify requirements risks Identify target users and stakeholders Define the scope and must-have features
24127445	Trương Hiển Long	PM	25%	Set up collaboration tools Present project schedule Present work package effort estimation
24127053	Trần Hoàng Minh Khang	Designer	25%	Preliminary Architecture Core Technology Stack Summary Identify technology risks Propose the hosting / deployment environmen

2 Vision (Revised since PA2)

2.1 Preliminary Problem Statement

Overview

Teachers and students need a consistent way to organize courses, distribute learning materials, manage assignments, and follow classroom activities. These tasks may currently be handled through a combination of chat applications such as Zalo or Discord, shared spreadsheets, and email. Each tool supports part of the process, but information must be maintained across several locations. Vible addresses this fragmentation by providing a lightweight web platform for everyday teaching and learning activities.

Current teaching and learning process

In a workflow based on separate communication and document-sharing tools, a teacher first establishes a class communication channel and maintains a list of participating students. Learning materials are distributed as messages, links, or attachments. When materials change, the teacher shares another version or updates a shared resource, and students must identify which information is current.

Assignments follow a similar process. The teacher announces the instructions and deadline, while students submit their work through the designated channel or email. The teacher then matches submissions to students, checks submission times, and records results separately. Questions, announcements, and feedback may appear alongside unrelated messages, requiring both teachers and students to search previous conversations.

Specific difficulties

For teachers, this arrangement creates repeated administrative work. Class membership, learning materials, assignment instructions, and submissions are maintained separately. Checking whether a student has submitted an assignment may require comparing messages or emails against a spreadsheet. Changes to instructions or deadlines must also be communicated clearly across the

tools used by the class. As the number of courses and participants increases, maintaining consistent records becomes more difficult.

For students, the main difficulty is locating the information needed to complete course activities. Materials and instructions can become buried in conversation histories, and multiple versions of a document can cause confusion. Students may need to check several channels to identify upcoming deadlines, confirm submission instructions, or find feedback. A submission sent to the wrong location can also be difficult for the teacher to identify and track.

Course information also requires controlled access. Teachers should manage their own courses, while students should access the materials and activities of courses in which they are enrolled. Managing membership separately from shared resources makes these access boundaries harder to maintain consistently.

Intended improvement

Vible will bring course membership, materials, assignments, and submissions into one platform. Teachers will be able to create and manage courses, maintain rosters, publish materials, and create assignments with deadlines. Students will be able to access their enrolled courses and submit work against the corresponding assignments. Grading, feedback, and course discussions are secondary features, subject to the project's priorities and available development time.

The intended outcome is a clearer and more consistent course workflow. Materials and submissions will be associated with their courses and assignments, reducing the need to reconcile information across separate tools. Success will be assessed through the completion and testing of the Must-have workflows.

Design and implementation constraints

The application must support modern desktop and mobile browsers through a responsive interface. Its planned SPA/PWA delivery should make core activities accessible through a web browser, while access to shared course data and file transfers requires an internet connection. Permissions must be enforced by the backend so that access follows each user's role and relationship to a course.

Implementation follows the selected Nuxt and TypeScript stack, PostgreSQL database, Cloudflare hosting, and object storage for uploaded files. Backend dependencies must be compatible with

the deployment environment. The planned use of free services also limits available storage and computing resources, so file handling and resource usage must remain manageable. Development is constrained by the course schedule and the team's available effort. Authentication, course management, enrollment, materials, and assignments therefore take priority. Built-in live video conferencing, private real-time messaging, payment processing, and institution-wide administrative integrations remain outside the delivery scope.

2.2 Product Position Statement

Element	Content
For	Teachers and students
Who	People who need an intuitive online platform to host/manage classroom activities and distribute learning materials
Vible	Is an Online Teaching Platform (or Learning Management System)
That	Provides an all-in-one hub to create courses, distribute materials, submit assignments, and provide grading feedback.
Unlike	Fragmented / Unspecilised chat apps (Zalo/Discord) or outdated traditional LMS platforms (Moodle).
Our product	Delivers a frictionless, lightweight SPA/PWA experience with a modern user interface and no unnecessary overhead.

2.3 Stakeholders and Users

#	Stakeholder / user	Description	Main interests
1	Teacher / Instructor	End user	Wants an intuitive, easy-to-use online platform to set up classes, distribute materials, assignments, and easily provide feedback to students.
2	Student	End user	Wants a reliable portal to access course materials managed by their teachers.
3	Sponsor — course instructor (proposed)	Provides academic direction, reviews deliverables, and evaluates the project against course requirements.	A feasible scope, timely delivery, and working software supported by consistent documentation and testing evidence.
4	System Admin	Technical operator responsible for deployment, service configuration, and operational maintenance.	Reliable operation, secure configuration, controlled infrastructure access, and manageable resource usage.

System Admin responsibilities may be performed through deployment and infrastructure tools; this stakeholder entry does not introduce an additional application administration interface.

2.4 Scope and Key Features

ID	Feature	Description	Priority (M/S/C/W)
F01	Authentication & authorization	Verify users (login/registration) and enforce role-based access for Teachers and Students.	Must
F02	Course management	Allow teachers to create, update, and delete courses.	Must
F03	Enrollment & roster management	Allow teachers to add or remove students from course rosters.	Must
F04	Course material management	Allow teachers to post/delete course materials (announcements, text, links, files) and enrolled students to view/download them.	Must

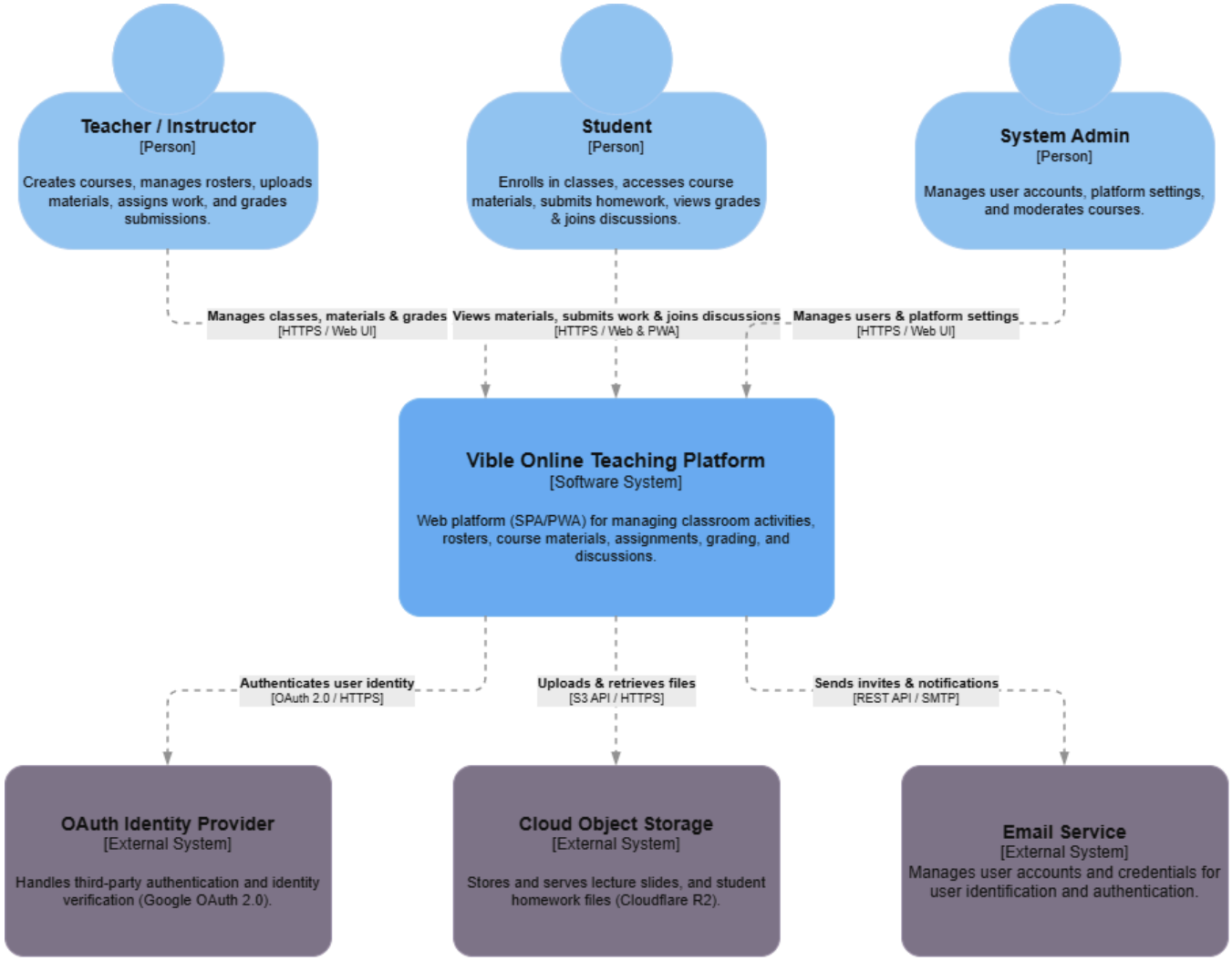
F05	Assignment & submission management	Allow teachers to create assignments with deadlines and enrolled students to submit their work.	Must
F06	Grading & feedback	Allow teachers to grade submitted assignments and provide written feedback to students.	Should
F07	Announcements & discussion channels	Provide course-level channels for class announcements and Q&A discussions between teachers and students.	Should
F08	Notifications & reminders	Send in-app or email alerts for new materials, upcoming assignment deadlines, and graded work.	Could
F09	Profile customization	Allow users to customize their profile details (display name, bio, avatar).	Could
F10	Private in-app messaging	Real-time 1-on-1 private chat between students or teachers.	Won't
F11	Payment gateway integration	Real-money course checkout, billing, and subscription management.	Won't
F12	Live video conferencing	Built-in live video/audio streaming or virtual classroom calls.	Won't

Out of scope:

1. Real-time communication (F10, F12): Live video conferencing (e.g., Zoom/WebRTC integration) and real-time private 1-on-1 in-app chat.
2. Complex institutional administrative integrations: Institution-wide user directory syncing (LDAP/SAML) and bulk administrative roster imports.
3. Real-money payment gateway integration (F11): Commercial course checkout and payment processing.

3 Proposed Solution

3.1 Preliminary Architecture



3.2 Technology Stack

3.2.1 Core Technology Stack Summary

Layer	Technology	Reason for choice
Frontend	Vue, NuxtUI, TypeScript, Tailwindcss	**Vue**: Delivers a fast, responsive Single-Page Application (SPA) and Progressive Web App (PWA).

		Nuxt UI, Tailwindcss: provide highly customisable pre-built components, **TypeScript:** for type safety.
Backend	Nodejs, Nuxt (Fullstack)	Nuxt supports Typescript natively, handles things such as configuration, rendering, routing and many more. Nuxt offers powerful modules such as nuxt-ui, pinia and test-utils.
Database	PostgreSQL	Follows ANSI SQL standards, making standard queries transferable.
Hosting / deployment	Cloudflare	There are official presets for building Nuxt app on Cloudflare
Testing & CI	Nuxt test utils, ESlint, Prettier, GitHub Actions	**ESLint and Prettier**: enforces consistent code styling, formatting standards, and syntax validation across the codebase. **test-utils**: integrates the official Nuxt testing module to manage unit, component, and end-to-end (E2E) test suites, supporting test runners including Vitest, Jest, Playwright, and Cucumber. **GitHub Actions**: automates workflow pipelines to enforce quality gates—executing linting, formatting checks, and automated test suites.

3.2.2 ORM Comparison Matrix

Criteria	Drizzle ORM (Primary Choice)	Prisma ORM (Alternative Choice)
Bundle Size & Architecture	Ultra-lightweight (~7.4 KB, zero binary dependencies)	Relies on a heavy Rust Query Engine or WASM adapter
Cloudflare Edge Compatibility	100% Native on Cloudflare Workers/Pages (V8 Isolates) out of the box	Requires Serverless Driver Adapters or Prisma Accelerate to run on Cloudflare Edge
Cold Start Latency	Near-zero cold start	Slightly higher cold start due to engine initialization
Query Control & SQL Transparency	SQL-like TypeScript syntax; full control over JOINs and indexes (ideal for PA3)	High-level declarative API; abstracts SQL (risk of hidden N+1 queries)
Pros	· Seamless deployment on Cloudflare Pages/Workers · Zero runtime overhead & full TypeScript inference	· Beginner-friendly declarative schema.prisma syntax · Built-in Prisma Studio GUI for data inspection

	· Transparent SQL control for PA3 index optimization	· Mature migration CLI
Cons	· Smaller third-party plugin ecosystem than Prisma · Requires familiarity with SQL concepts	· Friction when bundling for Cloudflare V8 Isolates · Larger deployment bundle size
Verdict for Vible	Selected (Primary): Optimal compatibility with Cloudflare Edge and PostgreSQL.	Selected (Fallback/Local Dev): Supported via driver adapters if declarative schema is preferred.

3.2.3 Hosting & Deployment Comparison Matrix

Criteria	Cloudflare Pages / Workers (Selected)	Vercel (Hobby Tier)	Render (Free Tier)
Runtime Architecture	Global Edge Network (V8 Isolates)	Serverless Functions + Edge	Container / Native Node.js
Nuxt 4 / Nitro Support	First-class (cloudflare-pages preset & NuxtHub)	First-class (vercel preset)	Standard Node.js build
Free Tier Quotas	Unlimited static bandwidth; 100,000 Worker req/day	100 GB bandwidth/mo; strict serverless execution limits	100 GB bandwidth/mo; 750 instance hrs/mo
Cold Start / Spin-down	0 ms cold start (always warm at Edge)	~500 ms – 2 s cold start on inactive API routes	30 – 50 s spin-down after 15 mins of inactivity
Latency in Vietnam	Ultra-low (Local Vietnam Edge PoPs)	Low–Moderate (Singapore region)	Moderate (Singapore region)
Storage Synergy	Cloudflare R2 (10 GB free, $0 egress fee for course files)	Vercel Blob (very small free quota)	Paid persistent disks; free PostgreSQL expires in 30–90 days
Pros	· Zero spin-down ensures reliable TA grading & PA5 live demo · Unlimited static bandwidth & fast Vietnam latency · Free R2 object storage mitigates PA0 file-upload risk	· Effortless GitHub PR preview deployments · Native Node.js serverless compatibility	· Runs full Node.js runtime without V8 Isolate limits

Cons	· V8 Isolates restrict native C++ Node.js modules · 10 ms CPU time limit per free request	· Strict non-commercial & function execution caps	· 30–50 s cold spin-down severely degrades UX during evaluation
Verdict for Vible	Selected: Best free quotas, zero cold-start delay, and R2 storage integration.	Rejected: Tighter serverless quotas and higher latency than Cloudflare.	Rejected: 30–50 s spin-down delay risks failing NFR performance targets.

3.2.4 Testing Framework Comparison Matrix

Criteria	@nuxt/test-utils + Vitest (Selected - Unit/Int)	Jest (Unit/Int Alternative)	Playwright (Selected - System E2E)
Target Test Levels	Unit Testing & Component / API Integration Testing	Unit & Integration Testing	Automated System (E2E) & Acceptance Testing
Nuxt 4 & Vite Compatibility	Official Nuxt module; natively resolves auto-imports, Pinia stores, and Vue 3 SFCs	Poor; requires complex Babel/ESM transforms and manual Nuxt mocks	Integrated into @nuxt/test-utils; manages Nuxt server lifecycle automatically
Execution Speed	Ultra-fast (shares Vite's in-memory transformation pipeline)	Slow (heavy CommonJS resolution and full rebuilds)	Fast (parallel headless browser contexts with auto-wait)
Coverage & Reporting	Built-in V8/Istanbul coverage (@vitest/coverage-v8) for PA5 reports	Built-in Istanbul coverage	HTML reports, trace viewer, screenshots, and video recording
Pros	· Zero-config compatibility with Nuxt 4 & TypeScript · Fast feedback loop in local dev and GitHub Actions CI · Directly generates PA5 statement/branch coverage metrics	· Mature ecosystem with extensive legacy documentation	· Directly fulfills PA5 requirement (2 Use Cases × 2 E2E scenarios) · Multi-browser support (Chromium, Firefox, WebKit) & OAuth flows
Cons	· Full Nuxt runtime environment takes a few seconds to boot	· High maintenance overhead with modern ESM and Vite projects	· Requires downloading browser binaries (~200 MB) in CI
Verdict for Vible	Selected (Unit/Integration): Official,	Rejected: Incompatible with Vite/Nuxt 4 without brittle workarounds.	Selected (System E2E): Best automation tool for PA4/PA5 system testing.

	fastest, and most reliable runner for Nuxt 4.		

3.3 Hardware and Operating Environment

Hardware: Personal laptops/PCs equipped with at least a 4-core CPU (Intel Core i5 / AMD Ryzen 5 / Apple M-series or higher), 16 GB RAM (8 GB minimum), and 20 GB of free SSD storage.

Operating System: Windows 10/11 (with WSL2), macOS 13+, or Linux (Ubuntu 22.04 LTS+).

Software & Local Tooling: Node.js (v20 LTS / v22 LTS), npm/pnpm package manager, Git, Visual Studio Code (with Vue Official, ESLint, Prettier, and Tailwind CSS IntelliSense extensions), Wrangler CLI (for local Cloudflare Pages/Workers emulation), and Docker Desktop / local PostgreSQL 16 instance.

Application Hosting: Cloudflare Pages & Workers running the compiled Nuxt 4 Nitro server bundle across global Edge V8 Isolates over HTTPS (TLS 1.3) with automated CI/CD previews linked to GitHub.

Database & Object Storage: Managed serverless **PostgreSQL** instance (e.g., Neon / Supabase with connection pooling) for relational data and **Cloudflare R2** object storage bucket for course materials and student submission files.

Client / End-User Operating Environment (Teachers, Students & Admins):

Devices: Desktop PCs, laptops, tablets, and smartphones (supported via responsive Tailwind CSS/Nuxt UI layouts and Progressive Web App capabilities).

Web Browsers: Any modern Evergreen web browser supporting HTML5, CSS3, and ES6+, including Google Chrome (v115+), Mozilla Firefox (v115+), Microsoft Edge (v115+), and Apple Safari (v16+).

Network: Standard broadband or mobile internet connection (Wi-Fi / 4G / 5G) with HTTPS access (port 443).

4 Project Plan (Revised since PA2)

4.1 Process and Phase Gates

The project follows the waterfall model. Each phase ends with a phase gate (a PA submission). After a PA is submitted it becomes a baseline; later changes must be updated in the document and listed in the next PA.

PA	Phase	Due	Exit criteria (fill in)
PA1	Initiation & planning	21/10/2026	**Vision:** Outlined the problem, identified stakeholders, and determined key functionality scope. **Solution:** Finalized C4 System Context design, defined the core tech stack, and specified system operational environments. **Project Plan:** Constructed a WBS, calculated cost estimates and assigned member roles and responsibilities. **Risk Management:** Identify project risks. **Tools & Collaboration Setup:** github code repository, created the project taskboard, set up discord server as the primary discussion channel, and established status meeting schedules.
PA2	Requirements	11/11/2026 (Extended)	**Requirements:** Defined functional, non-functional and domain requirments. **Use Case Model:** Finalized the complete use case diagram, defined actors and use case specifications. **Prototype / Mockups**: Present the screen list and wireframes/mockups for the main use cases **Traceability & Review:** Established the requirements matrix.

PA3	Design	25/11/2026	**Architectural Design:** Constructed the architectural design with required diagrams and specifications. **Object-Oriented Design:** Constructed object-oriented design with required diagrams and specifications. **Data Design:** Constructed data design with required diagrams and specifications. **UI/UX Design:** Identifed how users will interact with the application and provided required diagrams and specifications.
PA4	Test planning	02/12/2026	**Test Plan**: Defined testing scope, levels, environment, entry/exit criteria, and defect management. **Test Design**: Applied test design techniques (EP, BVA, and decision tables) to key features. **Test Cases**: Specified repeatable test cases for main workflows. **Traceability**: Mapped test cases to Must-have requirements in the traceability matrix.
PA5	Implementation, testing & deployment	16/12/2026	**Implementation:** Implemented Must-have features with reviewed source code and logged design deviations. **Testing & CI:** Executed test cases, recorded results and defects, and configured automated tests with CI. **Deployment & Working Software:** Deployed accessible working software and recorded the demo video. **Project Retrospective:** Completed plan-vs-actual evaluation, process reflection, and AI usage review.
Final	Presentation & demo	23/12/2026	

4.2 Work Breakdown Structure (WBS)

1. **Initiation and Planning**
   - 1.1. Vision
     - 1.1.1. Write the Preliminary Problem Statement
     - 1.1.2. Write the Product Position Statement
     - 1.1.3. Document Stakeholders and Users
     - 1.1.4. Define Scope and Key Features
   - 1.2. Proposed Solution
     - 1.2.1. Draw the Preliminary Architecture
     - 1.2.2. Document the Technology Stack
     - 1.2.3. Document Hardware and Operating Environment
   - 1.3. Project Plan
     - 1.3.1. Define Process and Phase Gates
     - 1.3.2. Create the Work Breakdown Structure
     - 1.3.3. Document Roles and Responsibilities
     - 1.3.4. Prepare the Schedule and Critical Path
     - 1.3.5. Prepare Work Package Estimation
     - 1.3.6. Prepare Human Resources and Cost Estimate
   - 1.4. Risk Management
     - 1.4.1. Identify project risks
     - 1.4.2. Calculate risk exposure
     - 1.4.3. Document risk responses and owners
   - 1.5. Tools and Collaboration Setup
     - 1.5.1. Set up communication channels
     - 1.5.2. Set up the task board
     - 1.5.3. Set up the repository and collaboration rules
     - 1.5.4. Document Tools and Collaboration Setup
2. **Software Requirements**
   - 2.1. Introduction and Context
     - 2.1.1. Write Purpose and Scope
     - 2.1.2. Prepare the Glossary

- 2.1.3. Elaborate the Problem Statement
- 2.1.4. Document Stakeholders

2.2. Requirements

- 2.2.1. Document Functional Requirements
- 2.2.2. Document Non-Functional Requirements
- 2.2.3. Document Domain Requirements and Business Rules
- 2.2.4. Define Acceptance Criteria
- 2.2.5. Prioritise requirements and record sources
- 2.2.6. Elicit and analyse requirements

2.3. Use Case Model

- 2.3.1. Identify Actors
- 2.3.2. Prepare the Use Case List
- 2.3.3. Draw the Use Case Diagram
- 2.3.4. Write Use Case Specifications

2.4. Prototype and Mockups

- 2.4.1. Prepare the Screen List
- 2.4.2. Create and check the Prototype and Mockups

2.5. Traceability and Review

- 2.5.1. Prepare the Requirements Traceability Matrix
- 2.5.2. Conduct and document the Requirements Review

3. **Software Design**

3.1. Architectural Design

- 3.1.1. Document Design Goals and Constraints
- 3.1.2. Document the Architectural Style
- 3.1.3. Update the System Context Diagram
- 3.1.4. Draw the Container Diagram
- 3.1.5. Draw the Component Diagram
- 3.1.6. Draw the Deployment Diagram
- 3.1.7. Write Architecture Decision Records

3.2. Object-Oriented Design

- 3.2.1. Draw the Domain Model

- 3.2.2. Draw the Class Diagram
- 3.2.3. Write Class Specifications
- 3.2.4. Draw Sequence Diagrams
- 3.2.5. Draw State Machine Diagrams
- 3.2.6. Document Design Principles and Patterns

3.3. Data Design

- 3.3.1. Draw the Data Diagram
- 3.3.2. Write Data Specifications
- 3.3.3. Document Normalisation and Indexes

3.4. User Interface and User Experience Design

- 3.4.1. Prepare the Screen List and Screen Flow
- 3.4.2. Write Screen Specifications
- 3.4.3. Complete the Accessibility Checklist

3.5. Traceability and Requirements Changes

- 3.5.1. Complete the Traceability Update
- 3.5.2. Document Changes to the Requirements Since PA2
- 3.5.3. Review design consistency

4. **Test Plan and Test Cases**

4.1. Test Plan

- 4.1.1. Define Objectives and Scope
- 4.1.2. Define Test Levels and Test Types
- 4.1.3. Select Test Design Techniques
- 4.1.4. Specify Test Environment and Test Data
- 4.1.5. Define Entry and Exit Criteria
- 4.1.6. Define Defect Management
- 4.1.7. Document Schedule, Responsibilities and Tools
- 4.1.8. Document Test Risks

4.2. Test Design

- 4.2.1. Define Equivalence Partitions and Boundary Values
- 4.2.2. Create Decision Tables

4.3. Test Cases

- 4.3.1. Prepare the List of Test Cases
- 4.3.2. Write Test Case Specifications
- 4.3.3. Complete Requirements–Test Traceability

5. **Implementation, Testing and Deployment**
   - 5.1. Development Setup
     - 5.1.1. Set up the coding environment
     - 5.1.2. Set up the testing environment
     - 5.1.3. Define coding standards
     - 5.1.4. Configure the CI pipeline
   - 5.2. Frontend Implementation
     - 5.2.1. Implement navigation and routing
     - 5.2.2. Implement page views and layouts
     - 5.2.3. Implement accessibility requirements
     - 5.2.4. Integrate frontend and backend APIs
   - 5.3. Database and File Storage
     - 5.3.1. Set up the database
     - 5.3.2. Configure database access
     - 5.3.3. Implement the database schema
     - 5.3.4. Seed test data
     - 5.3.5. Configure file storage
   - 5.4. Backend Implementation
     - 5.4.1. Implement authentication and authorisation (F01)
     - 5.4.2. Implement course management (F02)
     - 5.4.3. Implement enrollment and rosters (F03)
     - 5.4.4. Implement course materials (F04)
     - 5.4.5. Implement assignments and submissions (F05)
     - 5.4.6. Implement secondary features (F06–F09, optional)
   - 5.5. Testing and Code Quality
     - 5.5.1. Review code and resolve static analysis issues
     - 5.5.2. Write and execute unit tests
     - 5.5.3. Automate system test scenarios

- 5.5.4. Execute functional integration, system and acceptance tests
- 5.5.5. Fix defects and run regression tests
- 5.5.6. Implement Non-Functional Requirements
- 5.5.7. Execute non-functional tests
- 5.5.8. Verify test exit criteria

5.6. Deployment and Release

- 5.6.1. Configure the deployment environment
- 5.6.2. Deploy and verify the system
- 5.6.3. Prepare the Release and User Guide
- 5.6.4. Record and publish the Working Software Video
- 5.6.5. Prepare access details and demo accounts
- 5.6.6. Create the release tag

5.7. Implementation Report

- 5.7.1. Document Source Code and Structure
- 5.7.2. Update Feature Status
- 5.7.3. Document Coding Standards and Code Review
- 5.7.4. Document Static Analysis
- 5.7.5. Document Deviations from the Design

5.8. Testing and Deployment Report

- 5.8.1. Write the Test Execution Summary
- 5.8.2. Compile Test Case Results and the Defect List
- 5.8.3. Document Unit Tests and Coverage
- 5.8.4. Document Automated System Tests
- 5.8.5. Document Continuous Integration
- 5.8.6. Document the Deployment Environment

5.9. Project Retrospective

- 5.9.1. Document Plan vs. Actual
- 5.9.2. Document Process Reflection and Improvements
- 5.9.3. Document AI Usage Lessons

6. **Final Presentation and Demo**

6.1. Project Presentation

6.1.1. Prepare presentation slides

6.1.2. Prepare the live demo

6.1.3. Rehearse the presentation and Q&A

6.1.4. Deliver the presentation and demo

4.3 Estimation

- EJ = expert judgement.
- AN = analogy with an existing PA artifact, without claiming measured historical effort.
- TP = three-point estimate using E = (O + 4M + P) / 6; O/M/P values are person-hours.

WBS ID	Work package	Owner	Effort (person-hours)	Method / assumptions
1.1.1	Write the Preliminary Problem Statement	Lê Đoàn Nhật Huy	3	EJ; Preliminary business description and constraints.
1.1.2	Write the Product Position Statement	Lê Đoàn Nhật Huy	1	EJ; One product-position table.
1.1.3	Document Stakeholders and Users	Lê Đoàn Nhật Huy	1	EJ; Roles, interests and stakeholder table.
1.1.4	Define Scope and Key Features	Lê Đoàn Nhật Huy	3	EJ; MoSCoW feature list and scope boundaries.
1.2.1	Draw the Preliminary Architecture	Trần Hoàng Minh Khang	2	AN; One C4 context diagram.
1.2.2	Document the Technology Stack	Trần Hoàng Minh Khang	3	EJ; Compare and justify the selected stack.
1.2.3	Document Hardware and Operating Environment	Trần Hoàng Minh Khang	1	EJ; Development, test and runtime environments.
1.3.1	Define Process and Phase Gates	Huỳnh Trung Ngạn	2	EJ; Phase gates, dates and exit criteria.
1.3.2	Create the Work Breakdown Structure	Huỳnh Trung Ngạn	4	EJ; Decompose all PA deliverables and product work.
1.3.3	Document Roles and Responsibilities	Huỳnh Trung Ngạn	1	EJ; Main roles, backups and responsibilities.
1.3.4	Prepare the Schedule and Critical Path	Trương Hiển Long	4	EJ; Dependencies, CPM calculation and Gantt.

1.3.5	Prepare Work Package Estimation	Trương Hiển Long	4	EJ; Estimate all work packages and check totals.
1.3.6	Prepare Human Resources and Cost Estimate	Trương Hiển Long	1	EJ; Aggregate effort, labour rate and infrastructure costs.
1.4.1	Identify project risks	Trần Hoàng Minh Khang	2	EJ; Khang coordinates the consolidated risk list; earlier technology-risk and requirements-risk allocations were Khang and Huy respectively.
1.4.2	Calculate risk exposure	Lê Đoàn Nhật Huy	1	EJ; Define scales and calculate risk exposure; related-scope assignment based on Huy's earlier requirements-risk allocation.
1.4.3	Document risk responses and owners	Lê Đoàn Nhật Huy	2	EJ; Record mitigation, contingency and owners; related-scope assignment based on Huy's earlier requirements-risk allocation.
1.5.1	Set up communication channels	Huỳnh Trung Ngạn	1	EJ; Configure the team channel and access.
1.5.2	Set up the task board	Huỳnh Trung Ngạn	2	EJ; Create the board and populate work packages.
1.5.3	Set up the repository and collaboration rules	Trương Hiển Long	2	EJ; Repository, protection rules and review workflow.
1.5.4	Document Tools and Collaboration Setup	Trương Hiển Long	1	EJ; Record tool links and collaboration arrangements.
2.1.1	Write Purpose and Scope	Huỳnh Trung Ngạn	1	AN; Refine the approved PA1 scope.
2.1.2	Prepare the Glossary	Trương Hiển Long	1	EJ; Define project terms consistently.
2.1.3	Elaborate the Problem Statement	Huỳnh Trung Ngạn	2	AN; Elaborate the PA1 business description.
2.1.4	Document Stakeholders	Huỳnh Trung Ngạn	1	AN; Update the PA1 stakeholder table.
2.2.1	Document Functional Requirements	Lê Đoàn Nhật Huy	4	EJ; Specify the prioritised feature requirements.

2.2.2	Document Non-Functional Requirements	Trương Hiển Long	3	EJ; Define measurable quality requirements and verification.
2.2.3	Document Domain Requirements and Business Rules	Lê Đoàn Nhật Huy	3	EJ; Permissions, enrollment and submission policies.
2.2.4	Define Acceptance Criteria	Huỳnh Trung Ngạn	3	EJ; Given/When/Then criteria for Must-have features.
2.2.5	Prioritise requirements and record sources	Huỳnh Trung Ngạn	2	EJ; Confirm priorities and requirement sources.
2.2.6	Elicit and analyse requirements	Lê Đoàn Nhật Huy	4	EJ; Gather needs, analyse scope and resolve ambiguities.
2.3.1	Identify Actors	Trần Hoàng Minh Khang	1	AN; Refine the actors identified in PA1.
2.3.2	Prepare the Use Case List	Lê Đoàn Nhật Huy	2	EJ; List use cases and related requirements.
2.3.3	Draw the Use Case Diagram	Trần Hoàng Minh Khang	2	EJ; Actor relationships and use-case relationships.
2.3.4	Write Use Case Specifications	Trương Hiển Long	8	TP: O=4, M=8, P=12; Main, alternative and exception flows for key use cases.
2.4.1	Prepare the Screen List	Trần Hoàng Minh Khang	1	EJ; Map screens to use cases.
2.4.2	Create and check the Prototype and Mockups	Trần Hoàng Minh Khang	8	TP: O=4, M=8, P=12; Mockups, prototype links and accessibility checks.
2.5.1	Prepare the Requirements Traceability Matrix	Huỳnh Trung Ngạn	3	EJ; Link requirements, use cases and screens.
2.5.2	Conduct and document the Requirements Review	Lê Đoàn Nhật Huy	4	EJ; Four members, one hour each; review and resolution record.
3.1.1	Document Design Goals and Constraints	Trần Hoàng Minh Khang	2	EJ; Extract architectural drivers from PA2.
3.1.2	Document the Architectural Style	Trần Hoàng Minh Khang	2	EJ; Justify the selected architectural style.
3.1.3	Update the System Context Diagram	Trần Hoàng Minh Khang	2	AN; Refine the PA1 C4 context diagram.

3.1.4	Draw the Container Diagram	Trần Hoàng Minh Khang	3	EJ; Application, API, database and storage containers.
3.1.5	Draw the Component Diagram	Trần Hoàng Minh Khang	4	TP: O=2, M=4, P=6; Decompose the main application containers.
3.1.6	Draw the Deployment Diagram	Trương Hiển Long	3	EJ; Map runtime services and deployment connections.
3.1.7	Write Architecture Decision Records	Trần Hoàng Minh Khang	4	EJ; At least three decisions with alternatives and consequences.
3.2.1	Draw the Domain Model	Lê Đoàn Nhật Huy	4	EJ; Model core business entities and relationships.
3.2.2	Draw the Class Diagram	Trần Hoàng Minh Khang	6	TP: O=3, M=6, P=9; Classes, operations and relationships.
3.2.3	Write Class Specifications	Trương Hiển Long	6	EJ; Specify the six to eight key classes.
3.2.4	Draw Sequence Diagrams	Huỳnh Trung Ngạn	6	TP: O=3, M=6, P=9; Three to five key use cases with alternative behaviour.
3.2.5	Draw State Machine Diagrams	Huỳnh Trung Ngạn	4	EJ; Model relevant object lifecycles.
3.2.6	Document Design Principles and Patterns	Trương Hiển Long	2	EJ; Explain selected principles and patterns in the design.
3.3.1	Draw the Data Diagram	Trương Hiển Long	4	TP: O=2, M=4, P=6; Model stored entities and cardinalities.
3.3.2	Write Data Specifications	Trương Hiển Long	5	EJ; Table columns, types, keys and constraints.
3.3.3	Document Normalisation and Indexes	Trương Hiển Long	2	EJ; Verify normalisation and justify query indexes.
3.4.1	Prepare the Screen List and Screen Flow	Trần Hoàng Minh Khang	3	EJ; Identify screens and navigation paths.
3.4.2	Write Screen Specifications	Trần Hoàng Minh Khang	6	EJ; Controls, events and validation for key screens.
3.4.3	Complete the Accessibility Checklist	Lê Đoàn Nhật Huy	2	EJ; Check contrast, keyboard access, labels and responsiveness.
3.5.1	Complete the Traceability Update	Huỳnh Trung Ngạn	3	EJ; Map requirements to design elements.

3.5.2	Document Changes to the Requirements Since PA2	Lê Đoàn Nhật Huy	1	EJ; Record requirement changes, reasons and affected items.
3.5.3	Review design consistency	Trần Hoàng Minh Khang	4	EJ; Four members, one hour each; cross-check the design models.
4.1.1	Define Objectives and Scope	Lê Đoàn Nhật Huy	2	EJ; Define included and excluded testing scope.
4.1.2	Define Test Levels and Test Types	Trần Hoàng Minh Khang	2	EJ; Define test levels and applicable test types.
4.1.3	Select Test Design Techniques	Lê Đoàn Nhật Huy	2	EJ; Map requirements to suitable techniques.
4.1.4	Specify Test Environment and Test Data	Trương Hiển Long	3	EJ; Specify browsers, services, accounts and test data.
4.1.5	Define Entry and Exit Criteria	Lê Đoàn Nhật Huy	1	EJ; Define measurable entry and exit checks.
4.1.6	Define Defect Management	Huỳnh Trung Ngạn	1	EJ; Define severity, priority and defect workflow.
4.1.7	Document Schedule, Responsibilities and Tools	Huỳnh Trung Ngạn	1	EJ; Allocate test work, tools and dates.
4.1.8	Document Test Risks	Huỳnh Trung Ngạn	1	EJ; Record testing risks and responses.
4.2.1	Define Equivalence Partitions and Boundary Values	Trần Hoàng Minh Khang	4	EJ; Valid and invalid partitions with boundary values.
4.2.2	Create Decision Tables	Trần Hoàng Minh Khang	3	EJ; Decision rules for combined business conditions.
4.3.1	Prepare the List of Test Cases	Lê Đoàn Nhật Huy	2	EJ; List test cases, priorities and automation status.
4.3.2	Write Test Case Specifications	Lê Đoàn Nhật Huy	6	TP: O=3, M=6, P=9; Specify fifteen to twenty key test cases.
4.3.3	Complete Requirements–Test Traceability	Huỳnh Trung Ngạn	2	EJ; Verify coverage of every Must-have requirement.
5.1.1	Set up the coding environment	Trương Hiển Long	2	EJ; Install dependencies and verify local builds.

5.1.2	Set up the testing environment	Trương Hiển Long	2	EJ; Configure test runners and service connections.
5.1.3	Define coding standards	Huỳnh Trung Ngạn	1	EJ; Agree formatting and coding conventions.
5.1.4	Configure the CI pipeline	Trương Hiển Long	3	EJ; Build and test checks on pull requests.
5.2.1	Implement navigation and routing	Trần Hoàng Minh Khang	3	TP: O=1.5, M=3, P=4.5; Routes, navigation and role-aware page access.
5.2.2	Implement page views and layouts	Trần Hoàng Minh Khang	10	TP: O=5, M=10, P=15; Implement the approved page designs using existing UI components.
5.2.3	Implement accessibility requirements	Trần Hoàng Minh Khang	3	EJ; Keyboard, labels, focus and responsive behaviour.
5.2.4	Integrate frontend and backend APIs	Huỳnh Trung Ngạn	6	TP: O=3, M=6, P=9; Connect forms and actions to the backend.
5.3.1	Set up the database	Trương Hiển Long	1	EJ; Provision the selected database service.
5.3.2	Configure database access	Trương Hiển Long	1	EJ; Configure credentials and connection permissions.
5.3.3	Implement the database schema	Trương Hiển Long	4	TP: O=2, M=4, P=6; Implement the approved schema and indexes.
5.3.4	Seed test data	Lê Đoàn Nhật Huy	2	EJ; Create representative accounts and course data.
5.3.5	Configure file storage	Trương Hiển Long	2	EJ; Configure storage and application access.
5.4.1	Implement authentication and authorisation (F01)	Trương Hiển Long	6	TP: O=3, M=6, P=9; Registration, login, sessions and permission enforcement.
5.4.2	Implement course management (F02)	Huỳnh Trung Ngạn	4	TP: O=2, M=4, P=6; Course creation, updates and deletion.
5.4.3	Implement enrollment and rosters (F03)	Lê Đoàn Nhật Huy	4	TP: O=2, M=4, P=6; Join requests, approvals, withdrawal and roster management.

5.4.4	Implement course materials (F04)	Trương Hiển Long	6	TP: O=3, M=6, P=9; Course content with file upload and retrieval.
5.4.5	Implement assignments and submissions (F05)	Huỳnh Trung Ngạn	8	TP: O=4, M=8, P=12; Assignments, deadlines, submissions and late-submission rules.
5.4.6	Implement secondary features (F06–F09, optional)	Huỳnh Trung Ngạn	16	TP: O=8, M=16, P=24; Optional F06–F09 scope; excluded from the core total.
5.5.1	Review code and resolve static analysis issues	Huỳnh Trung Ngạn	4	EJ; Peer-review coordination and static-analysis fixes.
5.5.2	Write and execute unit tests	Trương Hiển Long	6	TP: O=3, M=6, P=9; Unit tests for core business logic.
5.5.3	Automate system test scenarios	Lê Đoàn Nhật Huy	5	EJ; At least two use cases with two scenarios each.
5.5.4	Execute functional integration, system and acceptance tests	Lê Đoàn Nhật Huy	6	EJ; Execute planned functional and acceptance scenarios.
5.5.5	Fix defects and run regression tests	Huỳnh Trung Ngạn	6	EJ; Allowance for defect fixes and regression checks.
5.5.6	Implement Non-Functional Requirements	Trương Hiển Long	5	EJ; Remaining approved quality requirements; exclude accessibility already estimated.
5.5.7	Execute non-functional tests	Lê Đoàn Nhật Huy	3	EJ; Verify applicable performance, security and usability targets.
5.5.8	Verify test exit criteria	Lê Đoàn Nhật Huy	1	EJ; Check execution, pass rates and unresolved defects.
5.6.1	Configure the deployment environment	Trương Hiển Long	2	EJ; Configure hosting, variables and service connections.
5.6.2	Deploy and verify the system	Trương Hiển Long	2	EJ; Deploy and smoke-test the hosted system.
5.6.3	Prepare the Release and User Guide	Trương Hiển Long	2	EJ; Release notes and build, configuration and user instructions.

5.6.4	Record and publish the Working Software Video	Trần Hoàng Minh Khang	3	EJ; Capture all Must-have flows and publish a video up to ten minutes.
5.6.5	Prepare access details and demo accounts	Lê Đoàn Nhật Huy	1	EJ; Verify demo accounts and access instructions.
5.6.6	Create the release tag	Trương Hiển Long	1	EJ; Tag the reviewed release revision.
5.7.1	Document Source Code and Structure	Trương Hiển Long	1	EJ; Record repository structure and architecture mapping.
5.7.2	Update Feature Status	Huỳnh Trung Ngạn	1	EJ; Record implemented, partial and omitted features.
5.7.3	Document Coding Standards and Code Review	Huỳnh Trung Ngạn	1	EJ; Summarise standards and review evidence.
5.7.4	Document Static Analysis	Huỳnh Trung Ngạn	1	EJ; Record analysis tools, findings and fixes.
5.7.5	Document Deviations from the Design	Trần Hoàng Minh Khang	1	EJ; Record implementation differences and reasons.
5.8.1	Write the Test Execution Summary	Lê Đoàn Nhật Huy	2	EJ; Summarise results against the PA4 exit criteria.
5.8.2	Compile Test Case Results and the Defect List	Lê Đoàn Nhật Huy	2	EJ; Compile existing results and defect records.
5.8.3	Document Unit Tests and Coverage	Trương Hiển Long	1	EJ; Compile unit-test and coverage evidence.
5.8.4	Document Automated System Tests	Lê Đoàn Nhật Huy	1	EJ; Record automated scenarios, scripts and outcomes.
5.8.5	Document Continuous Integration	Trương Hiển Long	1	EJ; Record pipeline stages and a successful run.
5.8.6	Document the Deployment Environment	Trương Hiển Long	1	EJ; Record hosting details and access information.
5.9.1	Document Plan vs. Actual	Huỳnh Trung Ngạn	2	EJ; Compare recorded scope, effort and schedule with the plan.
5.9.2	Document Process Reflection and Improvements	Huỳnh Trung Ngạn	2	EJ; Reflect on waterfall and possible Scrum improvements.

5.9.3	Document AI Usage Lessons	Lê Đoàn Nhật Huy	1	EJ; Summarise AI usage benefits, problems and lessons.
6.1.1	Prepare presentation slides	Everyone	6	EJ; Assemble fifteen to twenty slides from existing PA outputs.
6.1.2	Prepare the live demo	Trương Hiển Long	2	EJ; Prepare and verify the live demonstration scenario.
6.1.3	Rehearse the presentation and Q&A	Everyone	8	EJ; Four members, two hours each; rehearsal and Q&A preparation.
6.1.4	Deliver the presentation and demo	Everyone	2	EJ; Four members, thirty minutes each including setup, delivery and Q&A.

Core work: 353 person-hours.

(*) Optional F06–F09 work: 16 person-hours.

Total (including optional work): 369 person-hours.

4.4 Schedule

CPM uses calendar-day task durations and the FS, SS and FF dependencies, including lags. Forward and backward passes on the dependency network give a minimum duration of 58 days and 73 tasks with zero total float (TF = LS − ES = 0). The approved Gantt baseline spans 70 calendar days, from 15 October to 23 December 2026. Its planned phase and task dates are not imposed as constraints in this CPM calculation. SS and FF links allow overlapping work.

One representative critical path is:

1.1.1 Write the Preliminary Problem Statement → 1.1.4 Define Scope and Key Features → 1.2.1 Draw the Preliminary Architecture → 1.2.2 Document the Technology Stack → 1.3.5 Prepare Work Package Estimation → 1.3.4 Prepare the Schedule and Critical Path →

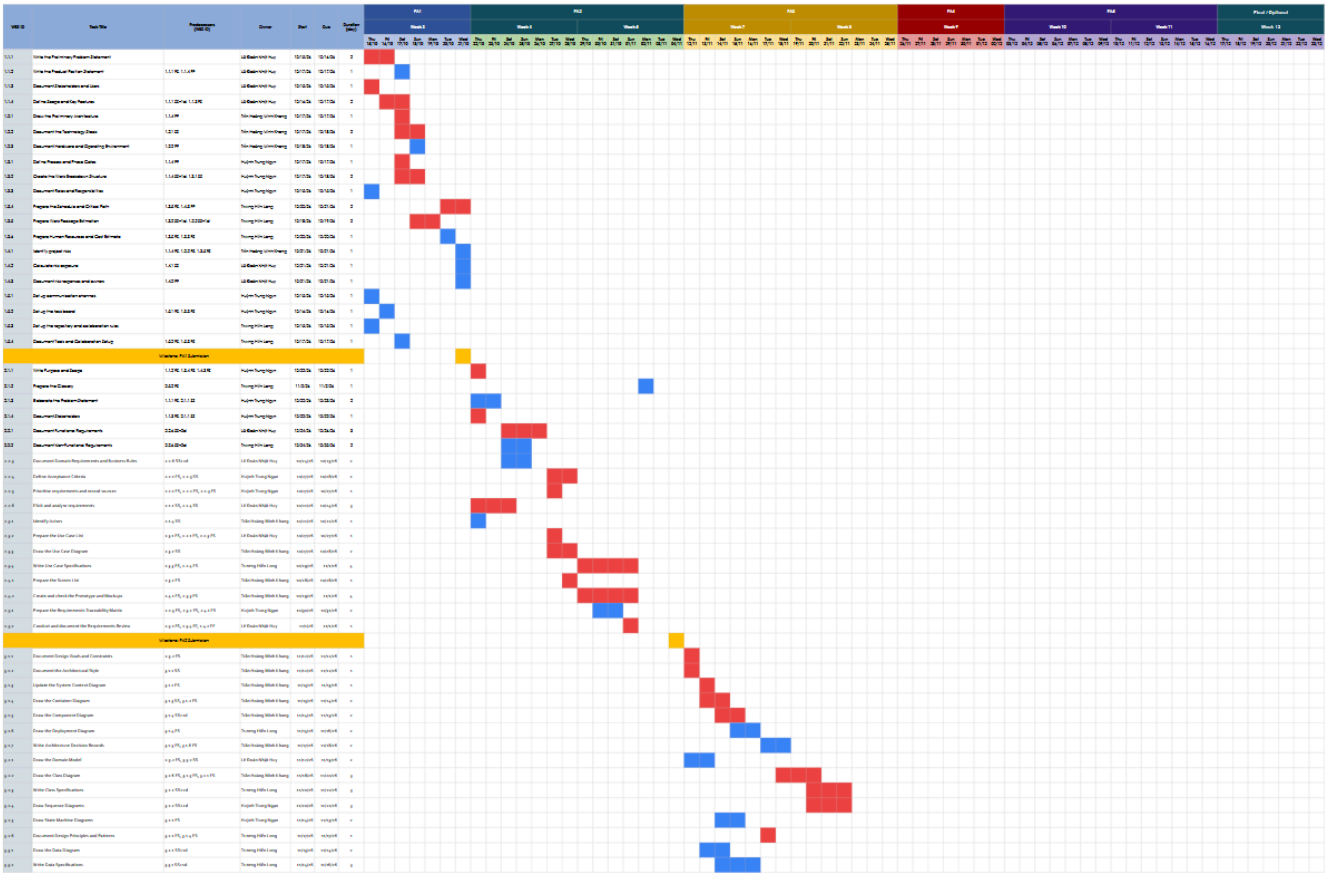
2.1.1 Write Purpose and Scope → 2.1.4 Document Stakeholders → 2.2.6 Elicit and analyse requirements → 2.2.1 Document Functional Requirements → 2.2.4 Define Acceptance Criteria → 2.3.4 Write Use Case Specifications → 2.5.2 Conduct and document the Requirements Review →

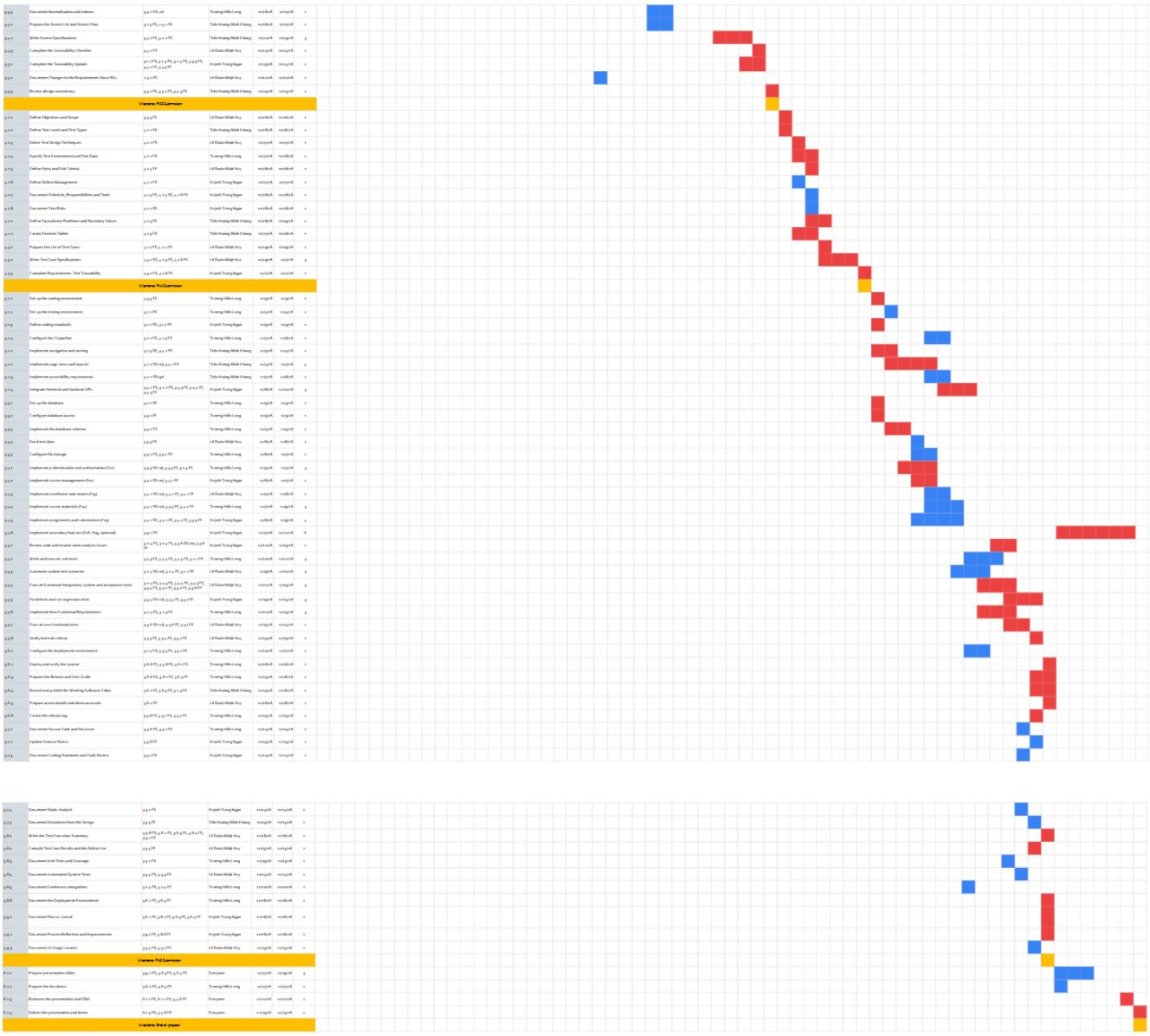
3.1.1 Document Design Goals and Constraints → 3.1.2 Document the Architectural Style → 3.1.4 Draw the Container Diagram → 3.1.5 Draw the Component Diagram → 3.2.2 Draw the Class Diagram → 3.2.3 Write Class Specifications → 3.5.1 Complete the Traceability Update → 3.5.3 Review design consistency →

4.1.1 Define Objectives and Scope → 4.1.2 Define Test Levels and Test Types → 4.1.3 Select Test Design Techniques → 4.2.1 Define Equivalence Partitions and Boundary Values → 4.3.1 Prepare the List of Test Cases → 4.3.2 Write Test Case Specifications → 4.3.3 Complete Requirements–Test Traceability →

5.1.1 Set up the coding environment → 5.1.3 Define coding standards → 5.2.1 Implement navigation and routing → 5.2.2 Implement page views and layouts → 5.2.4 Integrate frontend and backend APIs → 5.5.4 Execute functional integration, system and acceptance tests → 5.5.5 Fix defects and run regression tests → 5.5.8 Verify test exit criteria → 5.6.2 Deploy and verify the system → 5.6.3 Prepare the Release and User Guide → 5.9.1 Document Plan vs. Actual → 5.9.2 Document Process Reflection and Improvements → 5.4.6 Implement secondary features (F06–F09, optional) →

6.1.3 Rehearse the presentation and Q&A → 6.1.4 Deliver the presentation and demo.





find the original sheet @here

4.5 Roles and Responsibilities

Role	Main	Backup	Responsibilities
Team lead / Project manager	Huỳnh Trung Ngạn	Trương Hiển Long	Plan, assign tasks, weekly status report, contact point
Business analyst	Lê Đoàn Nhật Huy	Trần Hoàng Minh Khang	Elicit and document requirements, organise reviews

Designer / Architect	Trần Hoàng Minh Khang	Huỳnh Trung Ngạn	Architecture, class, data and UI design
Developers	Trương Hiển Long	Lê Đoàn Nhật Huy	Implement, unit test, review pull requests
Tester	Lê Đoàn Nhật Huy	Trần Hoàng Minh Khang	Test plan, test cases, system testing, test report

4.6 Human Resources and Cost Estimate

The estimate uses the effort in Section 4.3 and an assumed junior-developer rate of USD 5 per person-hour, applied to all project work.

Core effort is 353 person-hours; optional F06–F09 work adds 16 person-hours.

No paid subscriptions, hardware purchases or custom domain are budgeted.

Cost component	Basis	Estimated cost (USD)
Core project labour	353 person-hours × USD 5/hour	1,765
Tools and collaboration	GitHub Free, Figma Starter and open-source development/testing tools	0
Application hosting and file storage	Cloudflare Workers/Pages Free and R2 free allowance	0
Database hosting	Free PostgreSQL hosting tier assumed, such as Neon Free	0
Other costs	Reuse existing laptops and internet; use the provider's free subdomain	0
Core project total	**Labour + tools + hosting + other costs**	**1,765**
Optional F06–F09 labour	16 person-hours × USD 5/hour	80
Total including optional work	**USD 1,765 + USD 80**	**1,845**

5 Risk Management (Revised since PA2)

Probability is estimated from 0 to 1. Loss represents the severity of the impact if a risk occurs, using a scale from 1 (lowest) to 10 (highest). Ratings consider schedule, rework, deliverable quality, and the ability to complete Must-have features.

Loss	Impact
1–2	Minor disruption requiring small corrections within the current task.
3–4	Limited rework or delay that can be absorbed within the planned phase.
5–6	Significant rework or delay requiring tasks to be replanned or schedule buffers to be used.
7–8	Major disruption threatening a phase deadline or completion of Must-have features.
9–10	Critical disruption threatening project completion or delivery of a usable system.

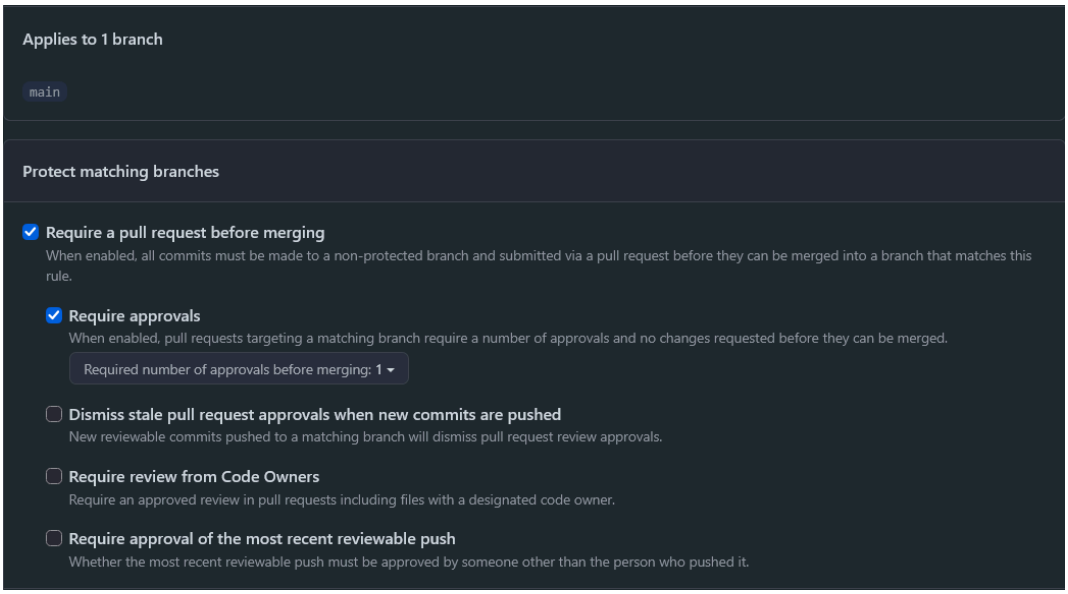
ID	Risk	Type	Probability	Loss	RE (P × L)	Mitigation / contingency	Owner
R1	Key member unavailable during PA5.	People	0.2	8	1.6	**Mitigation**: Maintain strict documentation, modular code, and PR reviews so knowledge is shared. **Contingency**: Reassign tasks to the designated backup role defined in Section 4.5	Huỳnh Trung Ngạn
R 2	Cloudflare Workers V8 Isolate limitations (10ms CPU limit on free tier, no native C++ Node.js modules) break backend dependencies.	Technology	0.3	7	2.1	**Mitigation**: Use Edge-compatible libraries (e.g., Drizzle ORM, Web Crypto API) and test builds early via wrangler. **Contingency**: Migrate deployment to Vercel or Render using standard Node.js runtime presets.	Trần Hoàng Minh Khang
R 3	Underestimation of effort required for	Estimation	0.4	5	2.0	**Mitigation**: Standardize modeling tools and review requirements early in PA2.	Lê Đoàn

	PA3 design models (UML/C4 diagrams).					**Contingency**: Reallocate development buffers to complete design tasks before coding begins.	Nhật Huy
R 4	Incomplete or changing requirements during PA2 affecting feature implementation in PA5.	Requirements	0.3	6	1.8	**Mitigation**: Conduct thorough BA reviews and prioritize features using MoSCoW methodology. **Contingency**: Descope Should/Could features to safeguard core Must-have deliverables.	Lê Đoàn Nhật Huy
R 5	Database performance degradation or slow queries under concurrent load during PA5 demonstration.	Technology	0.2	6	1.2	**Mitigation**: Design normalized relational schema and index frequently queried foreign key fields. **Contingency**: Implement query caching and optimize Drizzle ORM query execution plans.	Trương Hiển Long
R 6	Delayed team decisions or unclear coordination between analysis, design, development, and testing cause inconsistent deliverables and missed handoffs.	Organisational	0.3	7	2.1	**Mitigation:** Record decisions and task dependencies on the shared task board; review blockers and upcoming handoffs during weekly meetings. **Contingency:** Hold a focused coordination meeting, resolve conflicting decisions, and revise affected task sequencing to protect Must-have deliverables.	Project manager

6 Tools & Collaboration Setup (Revised since PA2)

The team must use:

- ✔ Moodle – announcements and PA submissions.
- ✔ Communication – **Discord**: https://discord.gg/g9xv36VByq
- ✔ Task board – **GitHub Projects:** https://github.com/users/nelan138/projects/3
- ✔ Git repository – **GitHub**: https://github.com/nelan138/vible-online-teaching-platform
  - main branch is protected; changes are merged through pull requests with at least one review



  - README (description, setup), .gitignore; never commit secrets (.env, API keys, passwords)
  - commit messages follow the Conventional Commits style (feat:, fix:, docs:, test:, ...)

✔ Weekly status meetings

- **Team meeting** (15 minutes) at 8:00 PM on Sundays
- **Weekly status report** (template provided) completed and committed to /docs/management/weekly every week before 11:59 PM on Tuesday following the team meeting.

Repository structure:

Folder	Content
/src	Source code
/docs/management	Project plan; weekly status reports in /docs/management/weekly
/docs/requirements	Vision, SRS, use cases, mockups
/docs/design	Architecture (C4), ADRs, UML models, UI design
/docs/test	Test plan, test cases, test reports
/docs/ai-usage	AI usage logs
/pa/PA1 … /pa/PA5	One sub-folder per PA submission (PDF + source files)

7 AI Usage Log

#	Member	Tool	Task / purpose	How the output was verified and changed
1	Trương Hiển Long	ChatGPT	Project Plan	Refined and strengthened ideas and report, clarified the proposed solution, and identified edge cases to challenge assumptions.
2	Huỳnh Trung Ngạn	Gemini	Help fill in 4.2 Work Breakdown Structure (WBS)	Content was written entirely by the author
3	Huỳnh Trung Ngạn	Gemini	Help fill in 4.1 Process and Phase Gates	Content was written entirely by the author
4	Trần Hoàng Minh Khang	Gemini	Draft comparison matrices (pros, cons, and technical criteria) for Database & ORM (PostgreSQL, MongoDB, Firebase, Drizzle, Prisma), Hosting (Cloudflare, Vercel, Render, Railway), and Testing (@nuxt/test-utils, Vitest, Jest, Playwright, Cypress) in Section 3.2.	Reviewed with Trương Hiển Long and Huỳnh Trung Ngạn; cross-checked technical constraints against our Nuxt 4 repository configuration (nuxt.config.ts, package.json) and finalized PostgreSQL + Drizzle/Prisma, Cloudflare, and Vitest.
5	Trần Hoàng Minh Khang	Gemini	Generate initial C4 System Context (Level 1) diagram structure (XML/Mermaid) and draft descriptions for Section 3.1 (Preliminary Architecture) and Section 3.3 (Hardware and Operating Environment).	Imported the diagram into draw.io (app.diagrams.net), manually refined the layout, colors, actors (Teacher, Student, System Admin), and external systems (Google OAuth, Cloudflare R2, Email Service) to match PA0 scope boundaries.
6	Trần Hoàng Minh Khang	Kimi	Brainstorm technology and organisational risks, probability/loss ratings, and mitigation strategies for Section 5 (Risk Management).	Verified and adjusted risk exposure scores and mitigation plans with the Project Manager (Huỳnh Trung Ngạn) to align with the 3-week implementation schedule and Cloudflare Edge runtime constraints.